

Module-A Report



CS432 Database Systems

INDIAN INSTITUTE OF TECHNOLOGY GANDHINAGAR

Palaj, Gandhinagar – 382355

B+ Tree-Based DBMS for Campus Trading

Implementation, Benchmarking and Visual Analysis

Submitted by

Team 8

Bhavik Patel, Hitesh Kumar, Jinil Patel, Pranav Patil, Sibtain

Contents

1	Introduction	2
1.1	Problem Addressed	2
1.2	Proposed Solution	2
2	Implementation	2
2.1	Code Structure	2
2.2	Node Layer (<code>node.py</code>)	3
2.3	B+ Tree Layer (<code>bplustree.py</code>)	3
2.3.1	Algorithmic View of Core Operations	3
2.3.2	Visualization Methods (<code>visualize_tree</code> family)	4
2.4	Complexity Summary	5
3	Performance Analysis	5
3.1	Benchmark Setup	5
3.2	Benchmark Findings	6
4	Visualisation	7
4.1	Stage-wise Tree Evolution (from Section 3.2 Notebook)	7
5	Conclusion	9
5.1	Project Summary	9
5.2	Challenges Faced	9
6	Work Distribution	10

1. Introduction

1.1 Problem Addressed

The Campus Trading application requires efficient data access over continuously growing datasets, including listings, members, offers, and transactions. Typical workloads include exact-key lookup (for example, by listing ID) and range-based filtering (for example, price windows). A linear baseline requires $O(n)$ scans, which becomes a bottleneck as the dataset grows. In practical terms, this directly affects user-facing tasks such as filtering listings by budget, loading searchable catalog views, and retrieving record details under frequent concurrent interactions.

From a systems perspective, the core challenge is to maintain **query responsiveness** while data volume grows. Without a structured index, performance degradation becomes unavoidable as every query tends toward full traversal. Therefore, an index design that supports both point and range access patterns is essential for this project context.

1.2 Proposed Solution

We implemented a B+ Tree-based indexing layer to provide stable and scalable DBMS-style performance. Under this design, point operations such as search, insert, update, and delete run in $O(\log n)$, while range operations run in $O(\log n + k)$. The linked-leaf structure supports ordered traversal directly, and balancing is preserved through split, borrow, and merge procedures during updates.

The proposed architecture is intentionally aligned with database indexing principles: internal nodes are used for routing only, while leaves hold data records in sorted order. This separation improves structural clarity, simplifies traversal logic, and enables **efficient range scans** with predictable asymptotic behavior.

2. Implementation

2.1 Code Structure

Key files used in Module A are listed below for clarity:

- `database/node.py`: defines `Node`, `LeafNode`, and `InternalNode`.
- `database/bplustree.py`: implements search/insert/delete/update/range query and visualization.
- `database/table.py`: schema-aware table wrapper on top of B+ Tree.
- `database/db_manager.py`: creates logical databases and manages tables.
- `database/benchmark.py`: compares B+ Tree vs brute-force baseline.

From an implementation perspective, the system is organized as a layered pipeline: node primitives define structural invariants, the B+ Tree layer enforces these invariants under updates, and the table/manager layers expose database-style APIs for CRUD and range filtering.

This layered approach also improves maintainability. The node layer encapsulates low-level invariants, the tree layer manages balancing semantics, and the table/manager layer maps these capabilities to application-facing data operations.

2.2 Node Layer (`node.py`)

Node class: stores shared metadata such as `keys`, `parent`, and order-derived constraints:

- `max_keys()` returns $m - 1$,
- `min_keys()` returns $\lceil m/2 \rceil - 1$,
- `is_full()` and `is_underflow()` enforce occupancy rules.

LeafNode:

- stores actual records in aligned `keys` and `values`,
- maintains `next` and `prev` pointers for sequential scans,
- provides `insert_at`, `remove_at`, `get_value`, `set_value` used by B+ Tree operations.

InternalNode:

- stores separator keys and child pointers,
- uses `find_child_index(key)` (binary-search style routing) to select the correct subtree,
- powers root-to-leaf navigation via `get_child_for_key`.

Algorithmically, this layer provides two guarantees on which the tree layer depends: (i) local occupancy constraints for each node, and (ii) deterministic routing to exactly one child for any search key.

2.3 B+ Tree Layer (`bplostree.py`)

2.3.1 Algorithmic View of Core Operations

Search Algorithm

1. Start from root.
2. Repeatedly route through internal separators until a leaf is reached.

3. Resolve key in the leaf array.

Insert Algorithm

1. Route to target leaf.
2. If key exists, overwrite value (upsert behavior).
3. Else insert in sorted order.
4. If node overflows, split node and propagate separator upward.
5. If propagation reaches root overflow, create a new root.

Delete Algorithm

1. Route to target leaf and remove key.
2. If node underflows, first try redistribution (borrow from siblings).
3. If redistribution fails, merge with sibling and remove parent separator.
4. Repeat upward while a parent violates occupancy constraints.

Together, these procedures guarantee that the tree remains **balanced**, all leaves remain at the same depth, and key ordering is preserved after each update workload.

Update

1. Locate target leaf using the same routing path as search.
2. Attempt in-place replacement of the value for the matching key.
3. Return success/failure flag without changing structure.

Range Query

1. Route to first leaf that may contain the lower bound.
2. Traverse leaf linked-list left-to-right.
3. Emit entries while keys are within $[start, end]$.
4. Stop immediately once key exceeds upper bound.

2.3.2 Visualization Methods (`visualize_tree` family)

The visualization pipeline in `bplustree.py` combines:

- `visualize_tree`: creates graph object and orchestrates rendering,
- `_add_nodes`: draws leaf/internal node labels,
- `_add_edges`: draws parent-child edges,

- `_add_leaf_links`: overlays dashed links for leaf-level ordering.

This makes structural properties visually verifiable during testing.

2.4 Complexity Summary

Table 1: Time Complexity of Main Operations

Operation	Complexity
<code>search(key)</code>	$O(\log n)$
<code>insert(key, value)</code>	$O(\log n)$
<code>delete(key)</code>	$O(\log n)$
<code>update(key, value)</code>	$O(\log n)$
<code>range_query(start, end)</code>	$O(\log n + k)$
<code>get_all()</code>	$O(n)$

The complexity profile confirms the intended design objective: operations scale logarithmically in tree size, while sequential extraction tasks remain linear in output size. This is especially appropriate for marketplace-style filtering and retrieval patterns.

3. Performance Analysis

3.1 Benchmark Setup

The benchmark compares B+ Tree and `BruteForceDB` for insertion, search, range query, and deletion across increasing dataset sizes.

Evaluation protocol used in the notebook:

1. Generate reproducible random key-value datasets for each size.
2. **Insert evaluation:** measure total insertion workload for both structures from an empty state.
3. **Search evaluation:** sample approximately 10% of inserted keys and measure point-lookup time.
4. **Range evaluation:** execute multiple random intervals and measure total range-query workload.
5. **Delete evaluation:** rebuild structures, delete a sampled fraction of keys, and measure total deletion workload.
6. Repeat over all configured sizes and store operation-wise curves.

All timings are collected in Python using high-resolution wall-clock timers and then plotted operation-wise for direct trend comparison.

To ensure a fair comparison, both structures receive equivalent datasets and equivalent operation counts per benchmark stage. For delete benchmarks, structures are reconstructed from the same seed-generated dataset before timing begins, preventing state carry-over effects from earlier measurements.

3.2 Benchmark Findings

The observed trends are consistent with theoretical complexity and remain stable across tested scales. Search shows the clearest widening performance gap as n increases, matching $O(\log n)$ behavior for B+ Tree versus $O(n)$ behavior for brute-force search. Range query performance also improves substantially because the B+ Tree performs a single logarithmic seek and then scans linked leaves sequentially. Insert and delete remain scalable despite occasional split/merge overhead, and this overhead does not alter asymptotic behavior.

Overall, the benchmark results support the key claim of this report: the B+ Tree index provides **better scalability** and **more predictable latency growth** than a linear baseline, particularly for high-volume lookup and filtering workloads.

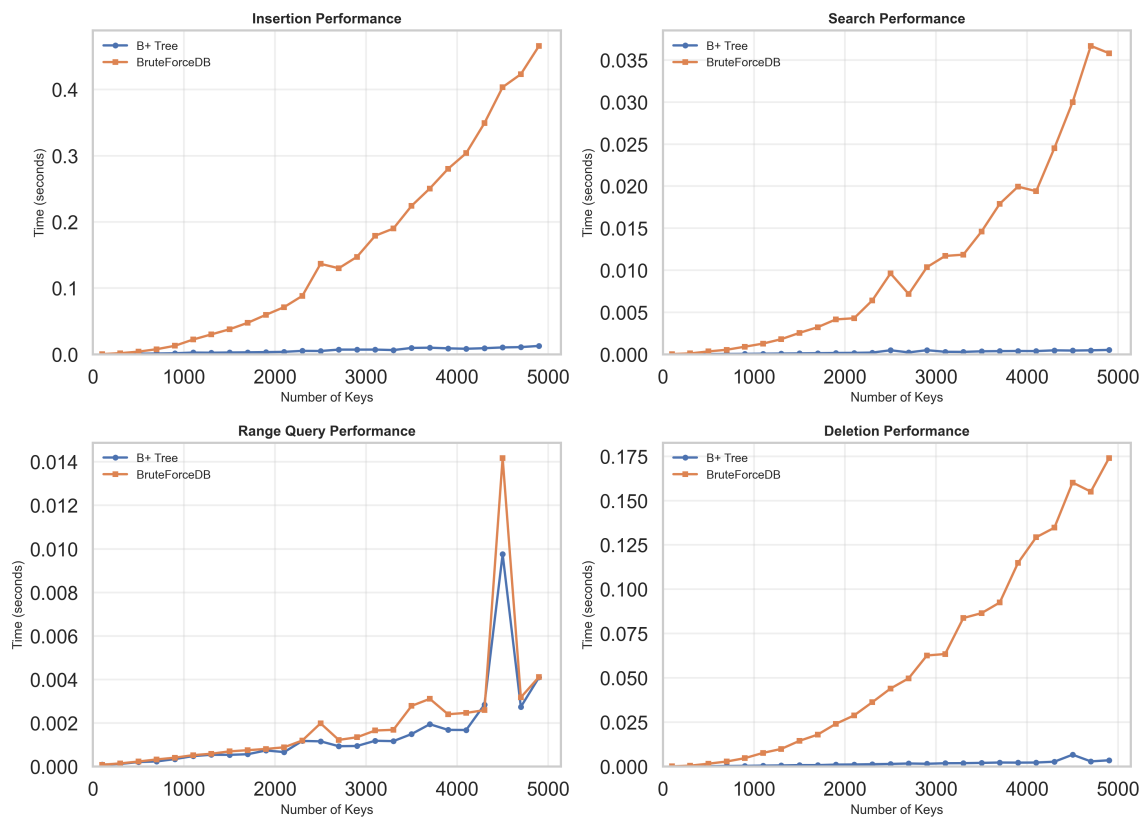


Figure 1: Combined operation-wise runtime comparison.

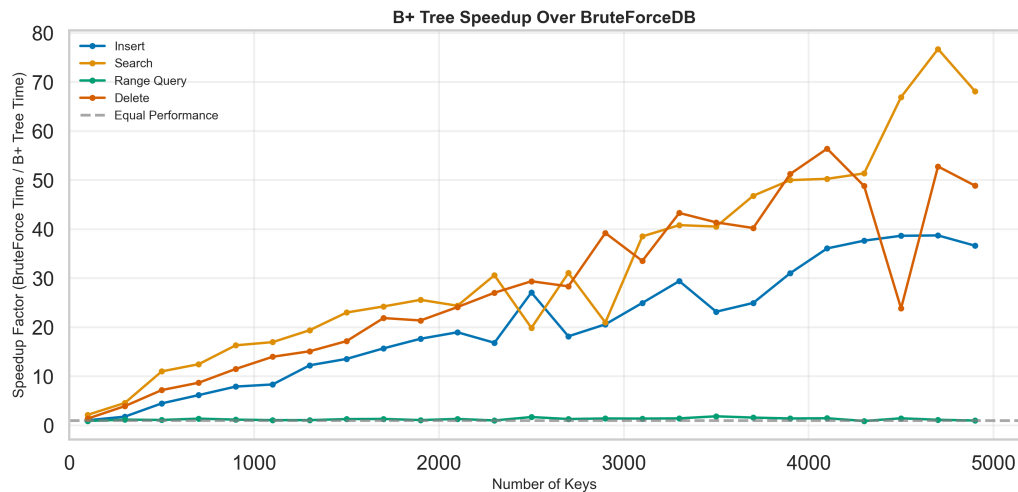


Figure 2: Speedup ratio (BruteForce time / B+ Tree time) across operation types.

4. Visualisation

The visualization outputs are used as structural correctness checks, not only as presentation artifacts. In particular, Section 3.2 of the notebook demonstrates staged structural evolution of the same tree under controlled insert and delete sequences.

These visual outputs are important because they provide direct evidence that balancing logic is functioning correctly under growth and shrinkage. In other words, the diagrams validate not just output correctness, but also **structural invariants** of the index.

4.1 Stage-wise Tree Evolution (from Section 3.2 Notebook)

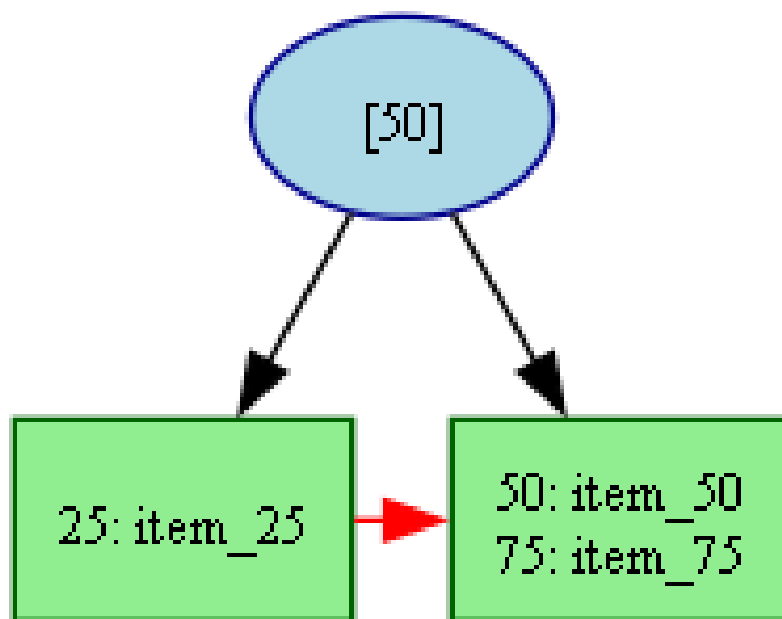


Figure 3: Stage 2: Tree after initial insertions [50, 25, 75].

Explanation: The tree remains shallow with a minimal internal routing structure; no deep split propagation is required at this stage.

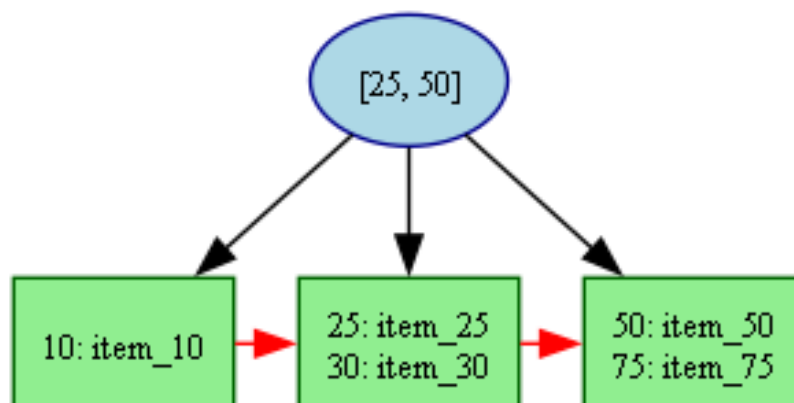


Figure 4: Stage 3: after inserting [10, 30], first visible split behavior.

Explanation: A leaf overflow triggers split and separator promotion, establishing clearer internal partitioning.

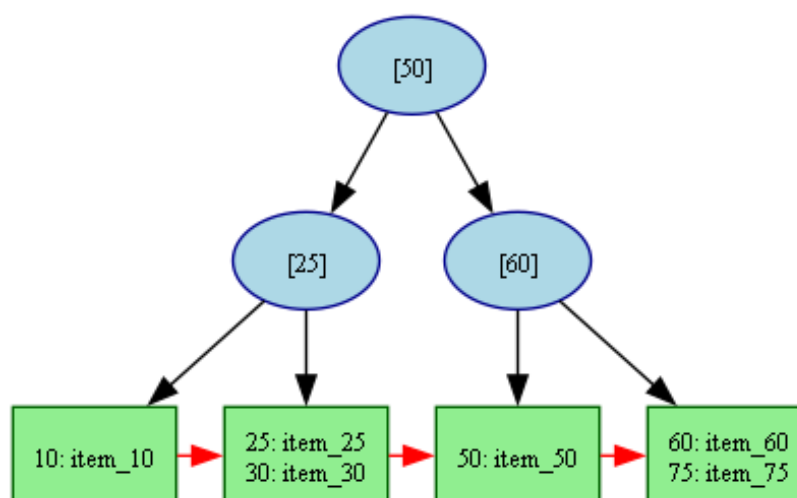


Figure 5: Stage 4: after additional insertion (60), further redistribution.

Explanation: Local balancing preserves sorted routing while preventing skew; the shape adjusts without violating occupancy constraints.

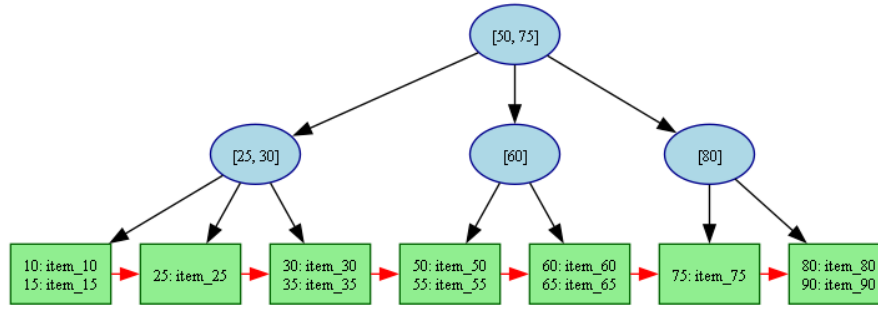


Figure 6: Stage 5: larger tree after additional insert workload.

Explanation: Height and fan-out increase in a controlled manner; all leaves remain at equal depth, preserving B+ Tree balance.

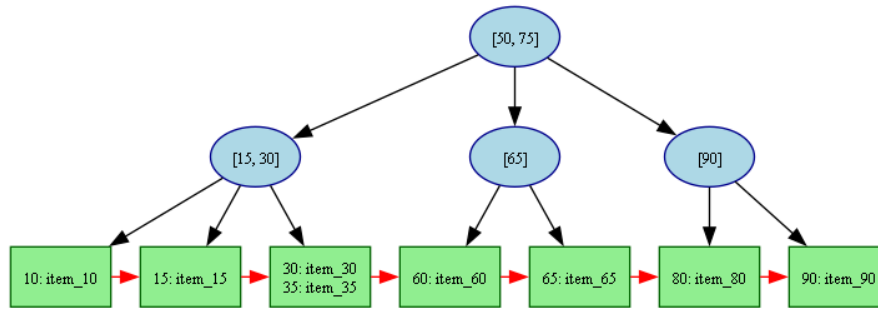


Figure 7: Stage 6: structure after deletion sequence and rebalancing.

Explanation: Delete-induced underflow is corrected through borrow/merge paths; the final shape remains valid and query-ready.

Taken together, the stage-wise sequence demonstrates stable structural behavior from initialization to split-heavy growth and then to deletion-driven rebalancing. This complements runtime benchmarking by confirming that performance is achieved without violating B+ Tree rules.

5. Conclusion

5.1 Project Summary

The Module A implementation demonstrates that B+ Tree indexing is a strong fit for Campus Trading workloads, particularly for mixed point-lookup and range-filter operations. By integrating balanced routing, linked-leaf traversal, and schema-aware table abstractions, the project delivers both algorithmic efficiency and practical usability.

5.2 Challenges Faced

The main implementation challenges were maintaining correct split propagation with parent pointer consistency, handling underflow robustly during deletion, and preserving leaf links dur-

ing structural changes. These concerns were central to keeping the tree both valid and efficient under mixed CRUD workloads.

6. Work Distribution

Table 2: Equal Work Distribution Across Team Members

Member	Primary Responsibility Set
Jinil Patel	B+ Tree core logic, split/merge validation, code review
Hitesh Kumar	Node abstractions, invariants, delete-path testing
Bhavik Patel	Benchmark framework, plots, performance interpretation
Pranav Patil	Table/DB manager integration, workflow demonstrations
Sibtain	Visualization pipeline, report writing, final consolidation

In addition to primary responsibility areas, all members participated in cross-review and validation of notebook outputs and report sections.